SLE-4: Final Viva & Architecture Decision Record

Course: 02AML204 – Introduction to Artificial Intelligence

SY B.Tech. CSE (AI & ML)

PRN	25UAM106	**Name**	Jui Jotiram Shinde
Division	B	**Date**	11 October 2026

PART A: Architecture Decision Record (ADR)

Title: Implementing and Comparing BFS and DFS for Graph Search

1. Context

The project searches a graph from a start node to a goal node. In SLE-1 I implemented the two classic uninformed search algorithms — Breadth-First Search (BFS) and Depth-First Search (DFS). In SLE-2 I profiled both algorithms on the identical graph (start node A) using Python's timeit module. Each goal was measured 10 times and the minimum, average and maximum times were recorded; nodes expanded were counted inside the search functions. Three goal positions were deliberately chosen: Goal B (best-case), Goal I (average-case) and Goal K (worst-case). BFS explores nodes level by level using a queue (FIFO). DFS explores one path as deeply as possible before backtracking, using a stack or recursion (LIFO).

I needed to formally record and justify the design decision to implement and compare both algorithms, and to connect the concrete performance evidence from SLE-2 with the system architecture prepared in SLE-3 (C4 Context, Container, Component and Code diagrams).

2. Decision

I decided to implement both Breadth-First Search (BFS) and Depth-First Search (DFS) and to compare them side-by-side on the same graph, the same start node and three carefully selected goal positions. The comparison metrics are average execution time (ms) and number of nodes expanded. This approach yields a fair, evidence-based comparison instead of assuming that one algorithm is always superior.

3. Alternatives Considered

- **BFS only:** Useful because it guarantees a shortest path (fewest edges) in an unweighted graph. However, it would give no direct comparison with DFS and would limit learning about the practical trade-offs between the two algorithms.
- **DFS only:** Can explore deep paths efficiently and often uses less memory than BFS. The major drawback is that it does not guarantee the shortest path in an unweighted graph.
- **A* search:** Uses a heuristic to guide the search and can be both complete and optimally efficient when the heuristic is admissible. It would, however, require designing, implementing and validating an additional heuristic function — work that was outside the intended scope of the early SLEs.

4. Rationale / Justification

My SLE-2 profiling produced clear numerical evidence on the same graph:

Test Case	BFS avg (ms)	DFS avg (ms)	BFS nodes	DFS nodes
Best • Goal B	0.019620	0.004310	2	2
Average • Goal I	0.020910	0.013620	9	6
Worst • Goal K	0.126880	0.022080	11	11

Observation: On this tested graph, DFS recorded lower average execution time in all three selected tests. Node expansion was equal in the best-case and worst-case tests, and lower for DFS in the average-case test (6 vs 9).

Py-spy flame graphs were also generated (BFS: 109 samples, DFS: 141 samples, both with 0 errors). These visual profiles confirm where execution samples were spent but are not used as the primary timing values. The measured results depend on graph structure, node ordering, goal position and implementation details (visited set, data structures). Different graphs may reverse the ranking; the numbers above describe only the experiment I performed.

The SLE-3 C4 architecture separates the overall system into Context, Container, Component and Code levels. This separation keeps input handling, the search logic (BFS and DFS modules) and result reporting clearly divided. Because the architecture already isolated the search component, it was straightforward to plug in both algorithms, feed them the same input and collect the same output metrics. Implementing both algorithms therefore aligns with both the measured evidence and the modular design prepared in SLE-3.

5. Consequences

Positive:

- I can compare both algorithms using consistent measures (average runtime and nodes expanded) on identical problem instances.

- The system is easier to explain, test and extend because of the clear C4 modular separation of Input → Search → Output.
- BFS can find a shortest path in an unweighted graph; DFS can be useful for deep exploration and often showed lower measured time on my graph.

Negative / Trade-offs:

- Implementing and maintaining two complete algorithms requires extra code and testing effort.
- BFS may need more memory for a wide frontier (queue), while DFS may return a non-shortest path.
- Runtime can vary across graphs and runs, so measurements must be repeated and interpreted carefully; results are not universal.

6. References

- GitHub repository: https://github.com/juishinde136-arch/IAI_SLE2_BFS_DFS
- My SLE-2 Performance Profiling Report (timeit averages, node counts, py-spy flame graphs) and SLE-3 C4 architecture diagrams (Context, Container, Component, Code).

PART B: Short Summary of My SLE Journey

In SLE-1 I created the BFS and DFS search code, prepared the README and maintained the AI Contribution Log. In SLE-2 I profiled both programs with Python's timeit module (10 runs per goal), counted nodes expanded for three goal positions (B, I, K), and generated py-spy flame graphs for visual confirmation. In SLE-3 I represented the complete system using C4 Context, Container, Component and Code diagrams, clearly separating input handling, the search modules and result reporting. In SLE-4 I recorded the decision to implement and compare both algorithms, grounding every claim in the concrete numbers from SLE-2 and the modular architecture from SLE-3.

This journey helped me connect four essential skills: writing correct search code, measuring performance rigorously, designing a clean system architecture, and explaining technical decisions clearly in writing and orally.

PART C: AI Contribution Note

AI tools used: Large language models as a learning and drafting aid.

What AI helped with: Understanding the SLE-2 profiling requirements, structuring the ADR according to the official guideline, improving clarity of wording, and suggesting possible viva questions.

What I did myself: All code was executed locally on my machine. The timing figures and node counts come from my own runs. I verified every number against my final SLE-2 report and every architectural claim against my SLE-3 diagrams. I will explain the algorithms, the measured results and the design decisions in my own words during the viva, and I will correct any detail that does not match my actual work.

PART D: Viva Preparation Notes (Detailed)

1. **What is BFS?** Breadth-First Search explores nodes level by level. It uses a queue (FIFO). In an unweighted graph it finds a shortest path measured by number of edges.
2. **What is DFS?** Depth-First Search explores as far as possible along one branch before backtracking. It uses a stack or recursion (LIFO). It does not guarantee the shortest path.
3. **What did your benchmark show?** On my graph DFS was faster in every case: Best-case Goal B (0.004310 ms vs 0.019620 ms), Average-case Goal I (0.013620 ms vs 0.020910 ms), Worst-case Goal K (0.022080 ms vs 0.126880 ms). Nodes expanded were equal in best and worst cases (2 and 11) and lower for DFS in the average case (6 vs 9).
4. **Why did you implement both algorithms?** To obtain a fair side-by-side comparison, to understand the practical trade-offs (path quality versus speed and memory), and because the C4 modular design already isolated the search component so both modules could be plugged in under identical conditions.
5. **What is C4 architecture?** C4 explains a system at four levels: Context (the system in its environment), Container (applications or services), Component (major internal parts) and Code (classes or functions). My diagrams separate Input → Search (BFS/DFS) → Output.
6. **What happens if the problem size becomes much larger?** BFS memory grows with the width of the frontier (queue). DFS may hit recursion-depth limits if implemented recursively. Both runtime and the number of nodes expanded will increase. I would re-run the same three-goal experiment on the larger graph and compare the new numbers.
7. **Key trade-off to state clearly:** BFS guarantees optimal path length (fewest edges) but can consume more memory; DFS may return a longer path but was faster and expanded equal or fewer nodes on the graphs I tested.
8. **How did you measure time and nodes?** Execution time was measured with Python's timeit module (10 runs per goal; min/avg/max recorded). Node expansion was counted manually inside the BFS and DFS functions. The same graph and start node A were used for both algorithms.
9. **What did AI contribute?** AI helped organise the ADR structure and refine wording. All measured numbers, code execution and final claims are my own responsibility; I verified them against my SLE-2 report and will answer viva questions from my own understanding.

10. **One-sentence summary of your decision:** I implemented and compared both BFS and DFS because the measured data showed meaningful differences and the C4 architecture already supported clean side-by-side testing of both algorithms.